

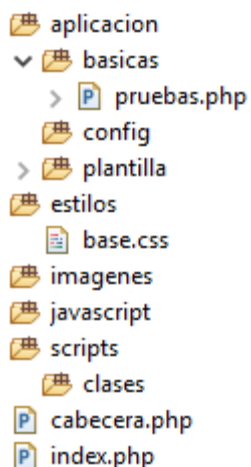
## 1.- DISEÑO DE LA ESTRUCTURA DE LA APLICACIÓN

El desarrollo de una aplicación implica unos costes que se deben minimizar para obtener el máximo rendimiento. Esto implica que operaciones como la creación de nuevas ventanas o la modificación de la interfaz (si fuera necesario) no supongan un coste elevado. Todo esto se puede conseguir mediante un diseño correcto de nuestra aplicación.

Si usamos un framework para desarrollar nuestra aplicación, el propio framework aporta los mecanismos para estructurar correctamente la aplicación.

Si no usamos un framework, lo mejor es preparar nuestra propia estructura que nos facilite la creación y mantenimiento de la aplicación.

En primer lugar creamos una estructura de carpetas y archivos. Como ejemplo podríamos tener una estructura semejante a:



Carpeta aplicación: En ella colocaremos los distintos ficheros de nuestra aplicación. Lo mejor es crear una carpeta en la que colocar todos los ficheros relacionados. Por ejemplo, para la gestión de artículos podemos crear la carpeta /aplicacion/articulos. Dentro de ella colocaremos los ficheros listarArticulos.php, nuevoArticulo.php, verArticulo.php, etc.

En aplicación también tenemos dos carpetas: config y plantilla. En config situamos archivos de configuración. Por ejemplo, un archivo con los datos de acceso a la base de datos. En plantilla podemos situar el/los ficheros necesarios para definir la plantilla del sitio web. Si tenemos una plantilla con la interfaz de la aplicación, será fácil modificarla en el futuro.

En la carpeta estilos colocaremos los distintos ficheros de estilos a usar.

En la carpeta imágenes colocaremos las imágenes que usáramos.

En la carpeta javascript podríamos colocar todos los ficheros de javascript que necesitáramos.

En la carpeta scripts colocaremos nuestros ficheros con aquella funcionalidad general. Por ejemplo, nuestras clases básicas necesarias para que funcione la aplicación pero que no implementa la lógica del programa (ACL, ControlAcceso, BD, Command, Sesiones, etc). Aquí podemos también colocar librerías externas que necesitemos como componentes para PDF, para convertir a EXCEL, ACCESS, etc.

Una vez creada la estructura de carpetas, el siguiente paso es crear la estructura de las páginas. Se debe intentar separar lo más posible la lógica de la aplicación de la interfaz. Para ello hay numerosas librerías que nos lo permiten hacer. Si no queremos usar ninguna api externa deberemos generar nuestra propia API para definir la interfaz.

Lo mejor es estudiar el diseño de página que deseamos y crear funciones (o una clase con métodos estáticos) que se encargen del diseño de la página. Como ejemplo podríamos tener el fichero /aplicación/plantilla/plantilla.php con la siguiente estructura.

```
<?php

function paginaError($mensaje)
{
    header("HTTP/1.0 404 $mensaje");
    inicioCabecera("PRACTICA");
    finCabecera();
    inicioCuerpo("ERROR");
    echo "<br />\n";
    echo $mensaje;
    echo "<br />\n";
    echo "<br />\n"; echo "<br />\n";
    echo "<a href='/index.php'>Ir a la pagina principal</a>\n";

    finCuerpo();
}

function inicioCabecera($titulo)
{
    ?>
    <!DOCTYPE html>
    <html lang="es">
        <head>
            <meta charset="utf-8">

            <!-- Always force latest IE rendering engine (even in intranet) &
            Chrome Frame
            Remove this if you use the .htaccess -->
            <meta http-equiv="X-UA-Compatible"
            content="IE=edge,chrome=1">

            <title><?php echo $titulo ?></title>
            <meta name="description" content="">
            <meta name="author" content="Administrador">

            <meta name="viewport" content="width=device-width; initial-
            scale=1.0">

            <!-- Replace favicon.ico & apple-touch-icon.png in the root of
            your domain and delete these references -->
            <link rel="shortcut icon" href="/favicon.ico">
            <link rel="apple-touch-icon" href="/apple-touch-icon.png">

            <link rel="stylesheet" type="text/css" href="/estilos/base.css">
        <?php
    }

function finCabecera()
{
    ?>
    </head>
    <?php
}

function inicioCuerpo($cabecera)
```

```

{
    global $acceso;
?>
<body>
    <div id="documento">

        <header>
            <h1 id="titulo"><?php echo $cabecera;?></h1>
        </header>

        <div id="barraLogin">

        </div>
        <div id="barraMenu">
            <ul>
                <li><a href="/index.php">Inicio</a></li>
            </ul>

        </div>

        <div>
<?php
}

function finCuerpo()
{
?>
            <br />
            <br />
        </div>
        <footer>
            <hr width="90%" />
            <div>
                &copy; Copyright by Profesor
            </div>
        </footer>
    </div>
</body>
</html>
<?php
}

```

Las funciones definidas son:

- inicioCabecera(\$titulo): Escribe las etiquetas correspondientes a <html> y <head>. Aquí se añaden enlaces a javascript y a hojas de estilo, es decir, todos aquellos elementos que queremos que se cargen en todas las páginas. Además se crea una etiqueta title con el \$titulo indicado
- finCabecera(). Cierra la etiqueta head
- inicioCuerpo(\$cabecera). Inicia el body e introduce una línea de cabecera (\$cabecera).
- finCuerpo(). Define un pie para las páginas y cierra las etiquetas body y html

Para completar la plantilla se tendrá el archivo con los estilos de nuestra página

(/estilos/base.css)

```
@charset "UTF8";

body {
    background-color: cyan;
    font-size: 16px;
}

#documento{
    margin: 0 auto;
    background-color: #8080ff;
    width: 700px;
}

#cabecera{
    background-color: #14e737;
    padding: 5px;
}

#cabecera #logo{
    width: 75px;
    height: 50px;
}

#cabecera #titulo{
    font-size: 1.3em;
    font-style: italic;
}

.boton{
    background-color: blue;
    color:white;
    border:thin black solid;
}

.boton a{
    text-decoration: none;
    color:white;
}

.boton a:visited{
    text-decoration: none;
    color:white;
}

.boton a:hover{
    background-color:cyan;
    color: black;
}

.boton:hover{
    background-color:cyan;
    color: black;
}

.tabla{
    width: 100%;
    border-collapse: collapse;
```

```
border: none;
background-color: white;
}

.tabla th{
background-color: teal;
color: blue;
}

.tabla tr.par{
background-color: black;
color: white;
}

.tabla tr.impar{
}

#pie{
background-color: #81e714;
color: blue;
font-size: 1.5 em;
padding: 5px;
}

.for_texto{
border:blue thin solid;
background-color: white;
padding: 2px;
margin:2px;
border-radius: 4px;
}

.error{
background-color: white;
color:red;
border:thin solid red;
padding:5px;
}
```

Se crean dos archivos más.

- **Cabecera.php:** este fichero se encarga de definir funciones básicas como la autocarga de clase o el modo de trabajo. Además se encarga de crear e inicializar todas las variables y clases necesarias para que trabaje mi aplicación.

```
<?php
define("RUTABASE", dirname(__FILE__));
//define("MODULO_TRABAJO","produccion"); //en "produccion o en desarrollo
define("MODULO_TRABAJO","desarrollo"); //en "produccion o en desarrollo

if (MODULO_TRABAJO=="produccion")
error_reporting(0);
else
error_reporting(E_ALL);
```

```

spl_autoload_register(function ($clase){
    $ruta=RUTABASE."/scripts/clases/";
    $fichero=$ruta.$clase.php";

    if (file_exists($fichero))
    {
        require_once($fichero);
    }
    else
    {
        throw new Exception("La clase $clase no se ha encontrado.");
    }
});

include(RUTABASE."/aplicacion/plantilla/plantilla.php");
include(RUTABASE."/aplicacion/config/acceso_bd.php");

//creo todos los objetos que necesita mi aplicación

```

- **Index.php:** Punto de acceso inicial a mi aplicación.

```

<?php
include_once(dirname(__FILE__) . "/cabecera.php");
//controlador

//dibuja la plantilla de la vista
inicioCabecera("APLICACION PRIMER TRIMESTRE");
cabecera();
finCabecera();
inicioCuerpo("2DAW APLICACION");
cuerpo(); //llamo a la vista
finCuerpo();
// *****

//vista
function cabecera()
{}

//vista
function cuerpo()
{
    ?>
    <br><br>
    Hola, estás en Index.php
<?php
}

```

Con esto se pretende aplicar el paradigma **Modelo-Vista-Controlador**. En nuestro caso, se separa la parte correspondiente a la vista (funciones de diseño de la página) del controlador (parte

inicial del fichero). En la vista se indica todo lo que se va a mostrar al usuario y en el controlador se hacen todas las operaciones y obtienen los datos a mostrar en la vista.

## 2.- ESTRUCTURA BÁSICA DE UNA PÁGINA

A partir del fichero anterior, una página cualquiera tendría la forma:

```
<?php
include_once(dirname(__FILE__)."/cabecera.php");

inicioCabecera("APLICACION PRUEBA");
cabecera();
finCabecera();

inicioCuerpo("APLICACION PRUEBA");
cuerpo();
finCuerpo();

// *****

function cabecera()
{

}

function cuerpo()
{

}
```

Para cada página particular se llaman a las funciones de la plantilla además de definir dos propias:

- cabecera(). Nos permite definir código html que se incluirá en la cabecera (head)
- cuerpo(). Se usa para definir el contenido propio de la página.

## 3.- PAGINA CON FORMULARIO

Otro punto a tener en cuenta es como actuar cuando se tiene un formulario. Se puede optar por:

- Un fichero para el formulario y otro para la acción del formulario
- Un fichero único con el formulario y la acción del mismo.

Todo parámetro que se reciba debe validarse y/o convertirse al tipo apropiado. Con ello evitamos posibles problemas tanto al almacenar en BD como con ataques por inyección de código.

A continuación muestro una página en la que se pide los datos de un nuevo artículo y se valida.

```
<?php
include_once(dirname(__FILE__)."/../../cabecera.php");

//inicializaciones
$datos=["nombre"=>"",
        "precio"=>0
    ];
$errores=[];

//comprobar si se ha dado a insertar
if ($_POST)
{
    $nombre="";
    if (isset($_POST["nombre"]))
    {
        $nombre=$_POST["nombre"];
        $nombre=strtolower(trim($nombre));
    }
    if ($nombre=="")
        $errores["nombre"][]="Debe indicarse un nombre";

    if (strlen($nombre)>20)
        $errores["nombre"][]="El nombre no puede tener mas de 30
caracteres";

    $datos["nombre"]=$nombre;

    $precio=0;
    if (isset($_POST["precio"]))
    {
        $precio=floatval($_POST["precio"]);
    }

    if ($precio==0)
        $errores["precio"][]="Debe indicarse un precio";

    $datos["precio"]=$precio;

    if (!$errores) //no hay errores hago la insercion
    {
        //codigo para el nuevo articulo
        $codigo=1; //codigo

        //se guarda el articulo

        //ir a ver el articulo insertado
        header("location: verArticulo.php?id=$codigo");
        exit;
    }
}
```



```

    }

}

inicioCabecera("PRUEBA: articulos");
cabecera();
finCabecera();

inicioCuerpo("NUEVO ARTICULO");
cuerpo($datos,$errores);
finCuerpo();

// *****

function cabecera()
{

}

function cuerpo($datos,$errores)
{

    ?>
    <br>
    <br>
    <br>

    <?php
    formulario($datos,$errores);
}

function formulario($datos,$errores)
{
    if ($errores)
    { //mostrar los errores
        echo "<div class='error'>";
        foreach($errores as $clave=>$valor)
        {
            foreach($valor as $error)
                echo "$clave => $error<br>".PHP_EOL;
        }
        echo "</div>";
    }

    ?>
    <form action="" method="post">
        <label for="nombre">Nombre: </label>
        <input type="text" name="nombre" id="nombre"
            value="<?php echo $datos["nombre"];?>" size=21
            maxlength="20"><br>

```

```
<label for="precio">Precio: </label>
<input type="text" name="precio" id="precio"
      value="<?php echo $datos["precio"];?>" size=4
      maxlength="3"><br>

<input type="submit" class="boton" value="Crear">
</form>
<?php
}
```

Para hacer el código más claro puede crearse una función que se encarga de validar el envío del formulario.

Deben usarse al mínimo las variables globales y dejarlas únicamente para variables que se pueden considerar casi “constantes” y que no se modifican.